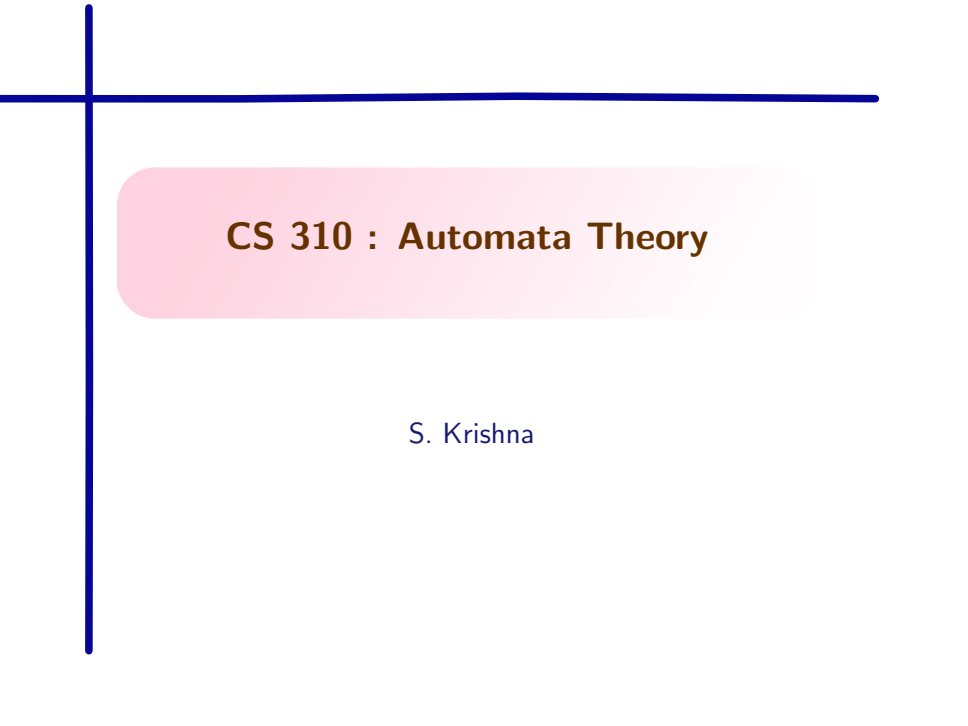


A decorative blue crosshair consisting of a vertical line and a horizontal line intersecting at the top-left corner of the slide.

CS 310 : Automata Theory

S. Krishna

CFGs to NPDAs : Correctness

Given $G = (N, T, P, S)$ be a CFG, construct PDA $A_G = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$, where

- ▶ $Q = \{q\}$, $q_0 = q$, $\Sigma = T$, $\Gamma = N \cup T$, $\perp = S$, $F = \emptyset$.

$$\delta(q, \epsilon, B) \triangleq \{(q, \beta) \mid B \rightarrow \beta \in P\}$$

$$\delta(q, a, a) \triangleq \{(q, \epsilon)\}$$

Lemma

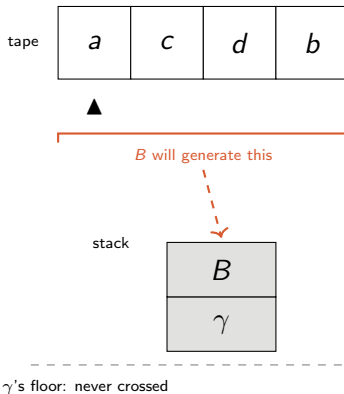
$L(G) = L(A_G)$, where acceptance is by empty stack.

- ▶ We have already seen $w \in L(G)$ implies $w \in L(A_G)$.
- ▶ Converse direction of correctness : From A_G to G . That is, whenever the constructed NPDA A_G accepts a word by empty stack, the word is generated in G . How to proceed?
- ▶ Let us relook thru an example again

Step 1: start

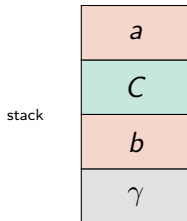
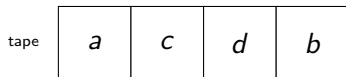
$B \rightarrow aCb$, $C \rightarrow cd$

start: B on top



Step 2: expand B

$B \rightarrow aCb, C \rightarrow cd$
expand $B \rightarrow aCb$ (ε)

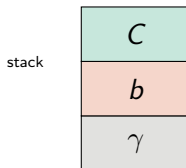
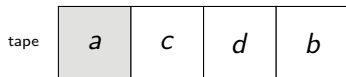


 γ 's floor: never crossed

Step 3: match a

$$B \rightarrow aCb, \quad C \rightarrow cd$$

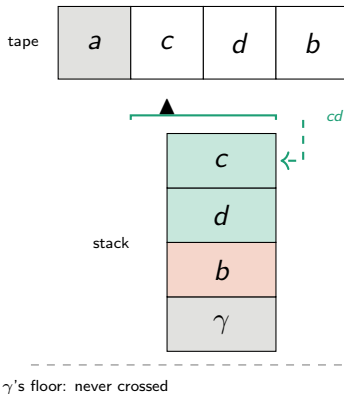
match a



 γ 's floor: never crossed

Step 4: expand C

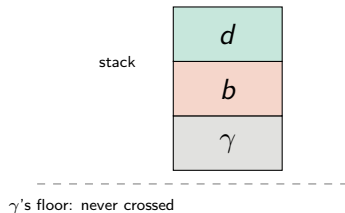
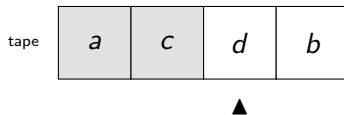
$B \rightarrow aCb, C \rightarrow cd$
expand $C \rightarrow cd (\varepsilon)$



Step 5: match c

$$B \rightarrow aCb, C \rightarrow cd$$

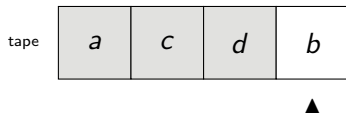
match c



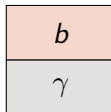
Step 6: match d

$$B \rightarrow aCb, \quad C \rightarrow cd$$

match d



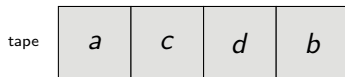
stack



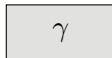
 γ 's floor: never crossed

Step 7: match b , done

$B \rightarrow aCb$, $C \rightarrow cd$
match b : done, back to γ



stack



 γ 's floor: never crossed

Conversion from CFG to NPDA

Prove that if a sequence of moves of A_G reads input x and pops a variable B from top of stack **without going below B on stack**, then in CFG G , $B \implies x$.

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .
- ▶ We induct on n , the number of moves in the computation $(q, x, X) \vdash^n (q, \epsilon, \epsilon)$.

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .
- ▶ We induct on n , the number of moves in the computation $(q, x, X) \vdash^n (q, \epsilon, \epsilon)$.
- ▶ **Base case** ($n = 1$): a single move pops X .

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .
- ▶ We induct on n , the number of moves in the computation $(q, x, X) \vdash^n (q, \epsilon, \epsilon)$.
- ▶ **Base case** ($n = 1$): a single move pops X .
 - ▶ If $X = a \in T$: $(q, a, a) \vdash (q, \epsilon, \epsilon)$. Then $\delta(q, a, a) = (q, \epsilon)$, so $x = a = X$.

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .
- ▶ We induct on n , the number of moves in the computation $(q, x, X) \vdash^n (q, \epsilon, \epsilon)$.
- ▶ **Base case** ($n = 1$): a single move pops X .
 - ▶ If $X = a \in T$: $(q, a, a) \vdash (q, \epsilon, \epsilon)$. Then $\delta(q, a, a) = (q, \epsilon)$, so $x = a = X$.
 - ▶ If $X = B \in N$: $(q, x, B) \vdash (q, \epsilon, \epsilon)$. Then $(q, x) \in \delta(q, \epsilon, B)$ for some $B \rightarrow x \in P$;

Conversion from CFG to NPDA: Correctness (converse)

We prove the converse: if A_G has a computation that reads $x \in T^*$ and pops a stack symbol X **without ever going below X** , then:

- ▶ if $X \in T$: $x = X$;
- ▶ if $X \in N$: $X \Rightarrow_{lm}^* x$ in G .
- ▶ We induct on n , the number of moves in the computation $(q, x, X) \vdash^n (q, \epsilon, \epsilon)$.
- ▶ **Base case** ($n = 1$): a single move pops X .
 - ▶ If $X = a \in T$: $(q, a, a) \vdash (q, \epsilon, \epsilon)$. Then $\delta(q, a, a) = (q, \epsilon)$, so $x = a = X$.
 - ▶ If $X = B \in N$: $(q, x, B) \vdash (q, \epsilon, \epsilon)$. Then $(q, x) \in \delta(q, \epsilon, B)$ for some $B \rightarrow x \in P$;
 - ▶ for the stack to become empty in this one move we need $x = \epsilon$, and no input is read. Then $B \rightarrow \epsilon$ gives $B \Rightarrow x$.

Conversion from CFG to NPDA: Correctness (converse), continued

- **Inductive step** ($n > 1$): then $X = B \in N$ (a terminal is always popped in a single move), and the first move must be

$$(q, x, B) \vdash (q, x, X_1X_2 \cdots X_k)$$

using some rule $B \rightarrow X_1X_2 \cdots X_k \in P$.

Conversion from CFG to NPDA: Correctness (converse), continued

- ▶ **Inductive step** ($n > 1$): then $X = B \in N$ (a terminal is always popped in a single move), and the first move must be

$$(q, x, B) \vdash (q, x, X_1 X_2 \cdots X_k)$$

using some rule $B \rightarrow X_1 X_2 \cdots X_k \in P$.

- ▶ Since the stack never goes below where B was, the remaining $n - 1$ moves must pop $X_1, \dots, X_k$ **one at a time, in order**. X_2 cannot be on top until X_1 is fully popped, and so on.

Conversion from CFG to NPDA: Correctness (converse), continued

- ▶ **Inductive step** ($n > 1$): then $X = B \in N$ (a terminal is always popped in a single move), and the first move must be

$$(q, x, B) \vdash (q, x, X_1 X_2 \cdots X_k)$$

using some rule $B \rightarrow X_1 X_2 \cdots X_k \in P$.

- ▶ Since the stack never goes below where B was, the remaining $n - 1$ moves must pop $X_1, \dots, X_k$ **one at a time, in order**. X_2 cannot be on top until X_1 is fully popped, and so on.
- ▶ So $x = x_1 x_2 \cdots x_k$ splits into k pieces, and the computation splits into consecutive stretches of $n_1, \dots, n_k$ moves ($n_1 + \cdots + n_k = n - 1 < n$) with

$$(q, x_i, X_i) \vdash^{n_i} (q, \epsilon, \epsilon), \quad i = 1, \dots, k.$$

Conversion from CFG to NPDA: Correctness (converse), continued II

- By the **induction hypothesis** (each $n_i < n$): $x_i = X_i$ if $X_i \in T$, and $X_i \Rightarrow_{lm}^* x_i$ if $X_i \in N$.

Conversion from CFG to NPDA: Correctness (converse), continued II

- ▶ By the **induction hypothesis** (each $n_i < n$): $x_i = X_i$ if $X_i \in T$, and $X_i \Rightarrow_{lm}^* x_i$ if $X_i \in N$.
- ▶ Substituting each derivation for the corresponding X_i into $B \rightarrow X_1 \cdots X_k$ gives a leftmost derivation

$$B \Rightarrow X_1 X_2 \cdots X_k \Rightarrow_{lm}^* x_1 x_2 \cdots x_k = x.$$

Conversion from CFG to NPDA: Correctness (converse), continued II

- ▶ By the **induction hypothesis** (each $n_i < n$): $x_i = X_i$ if $X_i \in T$, and $X_i \Rightarrow_{lm}^* x_i$ if $X_i \in N$.
- ▶ Substituting each derivation for the corresponding X_i into $B \rightarrow X_1 \cdots X_k$ gives a leftmost derivation

$$B \Rightarrow X_1 X_2 \cdots X_k \Rightarrow_{lm}^* x_1 x_2 \cdots x_k = x.$$

- ▶ This completes the induction. Taking $X = S$ and $n = \text{length of the accepting computation of } A_G \text{ on } w$ shows: $w \text{ accepted by } A_G \implies w \in L(G)$.

NPDA to CFG

NPDA to CFG : Notations

- ▶ Goal : Build a CFG which simulates the NPDA

NPDA to CFG : Notations

- ▶ Goal : Build a CFG which simulates the NPDA
- ▶ $[qBq']$: there is a sequence of moves from state q to q' that reads some w , starts with B on top of the stack, and in its last move, goes below where B was, never going below B at any earlier point.

NPDA to CFG : Notations

- ▶ Goal : Build a CFG which simulates the NPDA
- ▶ $[qBq']$: there is a sequence of moves from state q to q' that reads some w , starts with B on top of the stack, and in its last move, goes below where B was, never going below B at any earlier point.
- ▶ To capture what an NPDA accepts by empty stack, track $[q_0 \perp q]$ for any state q
- ▶ How to evolve $[q_0 \perp q]$ capturing the run of the NPDA?

Example NPDA

$$Q = \{q_0, q_1, q_2, q_3, q_4\}, \Sigma = \{a, e, f\}, \Gamma = \{\perp, X, Y, Z\}$$

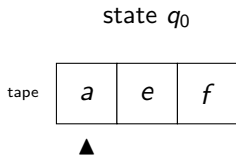
$$\delta(q_0, a, \perp) = \{(q_1, XYZ)\}$$

$$\delta(q_1, e, X) = \{(q_2, \epsilon)\}$$

$$\delta(q_2, \epsilon, Y) = \{(q_3, \epsilon)\}$$

$$\delta(q_3, f, Z) = \{(q_4, \epsilon)\}$$

Step 1: initial configuration



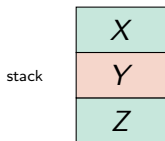
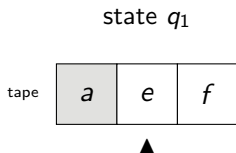
Bracket tracker

$[q_0 \perp ?]$

open – destination unknown

Read so far: (nothing yet)

Step 2: read a , pop $\perp$, push XYZ



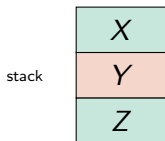
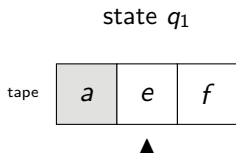
Bracket tracker

$[q_0 \perp ?]$

$[q_1 X q']$

Read so far: a

Step 2: read a , pop $\perp$, push XYZ



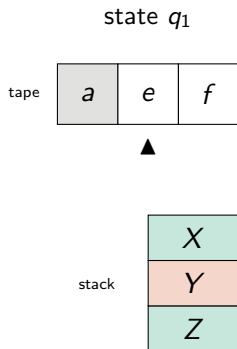
Bracket tracker

$[q_0 \perp ?]$

$[q_1 X q'] [q' Y q'']$

Read so far: a

Step 2: read a , pop $\perp$, push XYZ



Bracket tracker

$[q_0 \perp ?]$

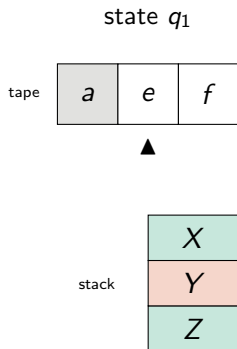
$[q_1 X q'] [q' Y q''] [q'' Z ?]$

All three are chained: X 's exit state q' is Y 's entry, Y 's exit q'' is Z 's entry.

Only the very last exit (which also closes the outer bracket) is still unknown.

Read so far: a

Step 2: read a , pop $\perp$, push XYZ



Bracket tracker

$[q_0 \perp ?]$

$[q_1 X q'] [q' Y q''] [q'' Z ?]$

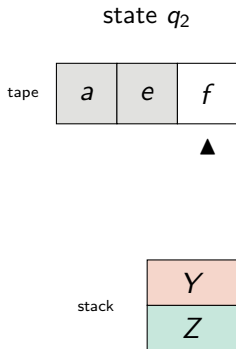
All three are chained: X 's exit state q' is Y 's entry, Y 's exit q'' is Z 's entry. Only the very last exit (which also closes the outer bracket) is still unknown.

$\perp$ becomes X, Y, Z :

$[q_0 \perp ?] \rightarrow a [q_1 X q_2] [q_2 Y q''] [q'' Z ?]$

Read so far: a

Step 3: read e , pop X



Bracket tracker

$$[q_1 X q_2] \rightarrow e$$

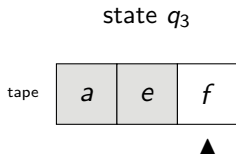
$$[q_2 Y q_3]$$

X 's bracket closes, with $q' = q_2$. Y 's opens next, from A we have $q'' = q_3$.

$$[q_0 \perp ?] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q ?]$$

Read so far: $a \cdot e$

Step 4: ϵ -move, pop Y



stack

Z

Bracket tracker

$[q_0 \perp ?]$

$[q_1 X q_2] \rightarrow e$

$[q_2 Y q_3] \rightarrow \epsilon$

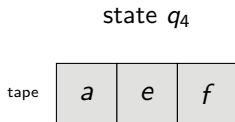
$[q_3 Z ?]$

Y 's bracket closes (no input read); Z 's opens next.

$[q_0 \perp ?] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z ?]$

Read so far: $a \cdot e \cdot \epsilon$

Step 5: read f , pop Z -stack empty



stack (empty)

Bracket tracker

$$[q_3 Z q_4] \rightarrow f$$

$$[q_0 \perp q_4]$$

Z 's bracket closes; stack is empty, so the outer bracket closes too – done!

$$[q_0 \perp q_4] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q_4]$$

Read so far: $a \cdot e \cdot \epsilon \cdot f = aef$

Putting it together

- ▶ The whole run: from q_0 with $\perp$ on top, we read aef and end in q_4 with the stack empty
- ▶ so $[q_0 \perp q_4] \xRightarrow{*} aef$
- ▶ Add $S \rightarrow [q_0 \perp q_4]$, so $S \xRightarrow{*} aef$.

Putting it together

- ▶ The whole run: from q_0 with $\perp$ on top, we read aef and end in q_4 with the stack empty
- ▶ so $[q_0 \perp q_4] \xRightarrow{*} aef$
- ▶ Add $S \rightarrow [q_0 \perp q_4]$, so $S \xRightarrow{*} aef$.
- ▶ Each piece of the run gives one nonterminal:

$$[q_0 \perp q_4] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q_4]$$

Putting it together

- ▶ The whole run: from q_0 with $\perp$ on top, we read aef and end in q_4 with the stack empty
- ▶ so $[q_0 \perp q_4] \xRightarrow{*} aef$
- ▶ Add $S \rightarrow [q_0 \perp q_4]$, so $S \xRightarrow{*} aef$.
- ▶ Each piece of the run gives one nonterminal:

$$[q_0 \perp q_4] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q_4]$$

- ▶ Each nonterminal $[q_i X_i q_{i+1}]$ covers the part of the run during which X_i is on top of the stack in state q_i , and derives whatever part of the input was read during that stretch without going below X_i , eventually entering q_{i+1} when we go below:

$$[q_1 X q_2] \rightarrow e \quad [q_2 Y q_3] \rightarrow \epsilon \quad [q_3 Z q_4] \rightarrow f$$

Putting it together

- ▶ The whole run: from q_0 with $\perp$ on top, we read aef and end in q_4 with the stack empty
- ▶ so $[q_0 \perp q_4] \xRightarrow{*} aef$
- ▶ Add $S \rightarrow [q_0 \perp q_4]$, so $S \xRightarrow{*} aef$.
- ▶ Each piece of the run gives one nonterminal:

$$[q_0 \perp q_4] \rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q_4]$$

- ▶ Each nonterminal $[q_i X_i q_{i+1}]$ covers the part of the run during which X_i is on top of the stack in state q_i , and derives whatever part of the input was read during that stretch without going below X_i , eventually entering q_{i+1} when we go below:

$$[q_1 X q_2] \rightarrow e \quad [q_2 Y q_3] \rightarrow \epsilon \quad [q_3 Z q_4] \rightarrow f$$

- ▶ Substituting: $[q_0 \perp q_4] \Rightarrow a [q_1 X q_2] [q_2 Y q_3] [q_3 Z q_4] \Rightarrow^* a \cdot e \cdot \epsilon \cdot f = aef$.